

中山大学计算机院本科生实验报告

(2025 学年春季学期)

课程名称：并行程序设计

批改人：

实验	CUDA 卷积优化	专业（方向）	计算机科学与技术
学号	22320021	姓名	陈安康
Email	1743826979@qq.com	完成日期	2025/6/25

1. 实验目的

练习 CUDA 编程，学习 CUDA 编程的原理以及 GPU 的架构对编程性能的影响。

2. 实验过程 and 核心代码

任务 1:

任务 1 代码如下，为了方便运算和处理，我将数据按照一维数组的形式进行存储，原始图像输入和 Kernel 都采用随机数进行初始化。其中所有的输出维度都被拼接成了一个一维数组，数组大小为 $\text{output_h} * \text{output_w} * \text{num_output_channels}$ 。而每一个线程负责每个通道中同一位置像素值的计算。并且这个实现使用的是全局内存。

```

__global__ void direct_convolution_kernel(const float* input, const float* kernel, float* output,
                                          int input_h, int input_w, int channels,
                                          int kernel_h, int kernel_w, int stride, int pad,
                                          int output_h, int output_w, int num_output_channels) {
    // 1. 计算当前线程负责的输出像素的全局坐标 (x, y)
    int out_x = blockIdx.x * blockDim.x + threadIdx.x;
    int out_y = blockIdx.y * blockDim.y + threadIdx.y;

    // 边界检查, 防止线程超出输出图像范围
    if (out_x < output_w && out_y < output_h) {
        for (int oc = 0; oc < num_output_channels; ++oc) { // 遍历输出通道
            float sum = 0.0f;

            // 2. 遍历卷积核的每一个元素
            for (int c = 0; c < channels; ++c) { // 遍历通道
                for (int kh = 0; kh < kernel_h; ++kh) { // 遍历卷积核高度
                    for (int kw = 0; kw < kernel_w; ++kw) { // 遍历卷积核宽度

                        // 3. 计算当前卷积核元素对应的输入图像坐标
                        int in_y = out_y * stride + kh - pad;
                        int in_x = out_x * stride + kw - pad;

                        // 4. 处理填充: 如果坐标在原始输入图像内, 则读取数据; 否则视为0
                        if (in_y >= 0 && in_y < input_h && in_x >= 0 && in_x < input_w) {
                            // 计算输入和卷积核中对应元素的索引
                            int input_idx = c * (input_h * input_w) + in_y * input_w + in_x;
                            int kernel_idx = oc * channels * kernel_h * kernel_w + c * kernel_h * kernel_w + kh * kernel_w + kw;
                            sum += input[input_idx] * kernel[kernel_idx];
                        }
                    }
                }
            }

            // 5. 将计算结果写入输出图像
            int output_idx = oc * (output_h * output_w) + out_y * output_w + out_x;
            output[output_idx] = sum;
        }
    }
}

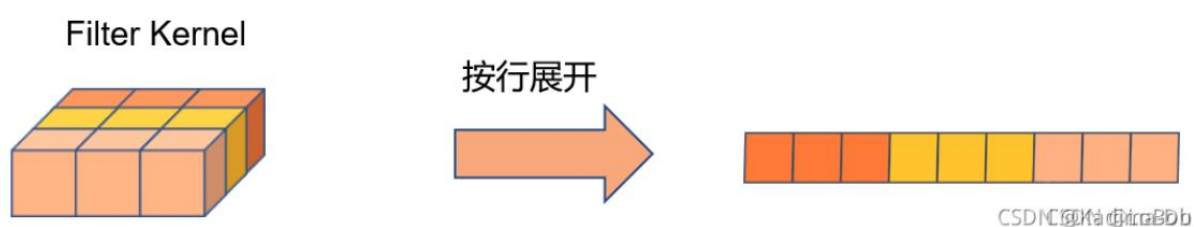
```

任务 2:

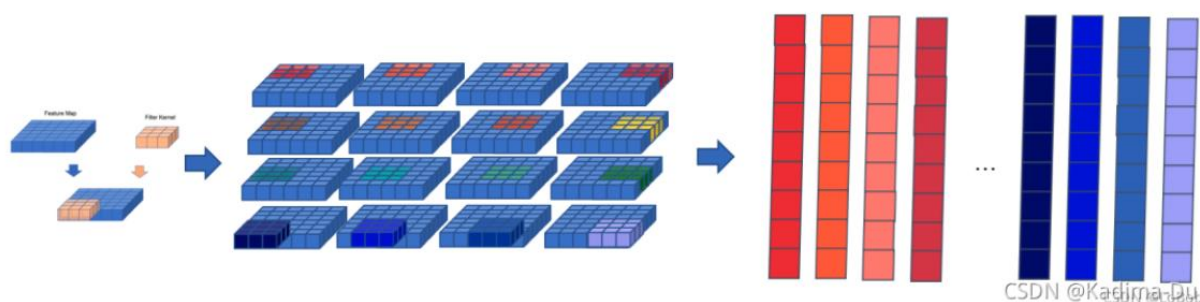
GEMM (General Matrix Multiplication - 通用矩阵乘法)即线性代数中学的标准矩阵乘法。对

于矩阵乘法的并行化已经十分成熟,为了让卷积算法转换成 GEMM,需要进行 im2col 操作。Im2col 是一种数据重排算法。它将卷积核区域的向量“拉直”成列向量,将每一步滑动窗口产生的列向量,并排拼接在一起,最终形成一个巨大的新矩阵。与此同时,所有的卷积核也被“拉直”,但它们是作为行向量,然后堆叠起来,形成一个卷积核矩阵。经过操作,原本的卷积操作就可以转换成矩阵乘法: $\text{Output Matrix} = \text{Kernel Matrix} \times \text{Col_Matrix}$ 。

- 展开Filter Kernel



- 展开计算过程中的特征图



实现如下,这里我分成两个阶段,第一个阶段使用 im2col 进行拉直,第二个阶段使用前面一个实验实现过的并行矩阵乘法进行运算得到最终结果。实现代码如下:

```

__global__ void im2col_kernel(float* col_matrix, const float* input,
                             int input_h, int input_w, int channels,
                             int kernel_h, int kernel_w, int stride, int pad,
                             int output_h, int output_w) {

    const int kernel_flat_size = kernel_h * kernel_w * channels;
    const int output_pixels = output_h * output_w;

    int row_idx = blockIdx.y * blockDim.y + threadIdx.y;
    int col_idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (row_idx >= kernel_flat_size || col_idx >= output_pixels) {
        return;
    }

    int oh = col_idx / output_w;
    int ow = col_idx % output_w;
    int c = row_idx / (kernel_h * kernel_w);
    int kh_rem = row_idx % (kernel_h * kernel_w);
    int kh = kh_rem / kernel_w;
    int kw = kh_rem % kernel_w;
    int in_y = oh * stride - pad + kh;
    int in_x = ow * stride - pad + kw;

    float val = 0.0f;
    if (in_y >= 0 && in_y < input_h && in_x >= 0 && in_x < input_w) {
        val = input[c * (input_h * input_w) + in_y * input_w + in_x];
    }

    col_matrix[row_idx * output_pixels + col_idx] = val;
}

```

```

__global__ void gemm_shared(const float* A, const float* B, float* C, int m, int n, int k) {
    int ty = threadIdx.y;
    int row = blockIdx.y * TILE_SIZE + ty;
    int col = blockIdx.x * TILE_SIZE + tx;

    float sum = 0.0f;
    for (int t = 0; t < (n + TILE_SIZE - 1) / TILE_SIZE; ++t) {
        if (row < m && (t * TILE_SIZE + tx) < n) {
            tile_A[ty][tx] = A[row * n + (t * TILE_SIZE + tx)];
        } else {
            tile_A[ty][tx] = 0.0f;
        }

        if ((t * TILE_SIZE + ty) < n && col < k) {
            tile_B[ty][tx] = B[(t * TILE_SIZE + ty) * k + col];
        } else {
            tile_B[ty][tx] = 0.0f;
        }
        __syncthreads();

        for (int i = 0; i < TILE_SIZE; ++i) {
            sum += tile_A[ty][i] * tile_B[i][tx];
        }
        __syncthreads();
    }

    if (row < m && col < k) {
        C[row * k + col] = sum;
    }
}

```

任务 3:

cuDNN，全称 NVIDIA CUDA® Deep Neural Network library，是 NVIDIA 公司专门为深度神经网络（Deep Neural Networks, DNN）打造的、基于 CUDA 的 GPU 加速基元库（Library of Primitives）。是一个介于底层 CUDA 和上层深度学习框架之间的、至关重要的加速中间件。它将 GPU 硬件的潜力，通过一套标准化的 API 接口，高效、稳定地提供给整个 AI 生态，是现代 GPU 驱动的人工智能革命的基石之一。它不仅采用高效的优化算法，还针对 cuda 底层进行优化加速，最大限度地压榨出每一代 GPU 架构）的性能。

使用 cuDNN 库执行卷积操作，其核心流程可概括为“配置”与“执行”两大阶段。在配置阶段，首先需初始化一个全局的 cuDNN 句柄（Handle），随后通过创建一系列“描述符”（Descriptor）对象，向 cuDNN 详细说明本次运算的所有信息：这包括输入/输出数据和卷积核的维度、数据类型与内存布局，以及卷积操作本身的步幅、填充等参数。此阶段还可以让 cuDNN 自动选择最快的卷积算法并为其准备所需的“工作空间”（Workspace）内存。配置完成后，进入执行阶段，只需调用单一的核心计算函数（如 cudnnConvolutionForward），cuDNN 便会利用这些配置信息在 GPU 上高效完成运算。最后，程序结束时需调用对应的销毁函数，释放所有创建的句柄和描述符等资源。核心代码实现如下：


```

cudnnHandle_t cudnn_handle;
CHECK_CUDNN_ERR(cudnnCreate(&cudnn_handle));

cudnnTensorDescriptor_t input_desc, output_desc;
cudnnFilterDescriptor_t kernel_desc;
CHECK_CUDNN_ERR(cudnnCreateTensorDescriptor(&input_desc));
CHECK_CUDNN_ERR(cudnnCreateTensorDescriptor(&output_desc));
CHECK_CUDNN_ERR(cudnnCreateFilterDescriptor(&kernel_desc));

CHECK_CUDNN_ERR(cudnnSetTensor4dDescriptor(input_desc, CUDNN_TENSOR_NCHW, CUDNN_DATA_FLOAT, n_images, input_c, input_h, input_w));
CHECK_CUDNN_ERR(cudnnSetFilter4dDescriptor(kernel_desc, CUDNN_DATA_FLOAT, CUDNN_TENSOR_NCHW, kernel_c_out, kernel_h, kernel_w));

cudnnConvolutionDescriptor_t conv_desc;
CHECK_CUDNN_ERR(cudnnCreateConvolutionDescriptor(&conv_desc));
CHECK_CUDNN_ERR(cudnnSetConvolution2dDescriptor(conv_desc, pad, pad, stride, stride, 1, 1, CUDNN_CROSS_CORRELATION));

int out_n, out_c, out_h, out_w;
CHECK_CUDNN_ERR(cudnnGetConvolution2dForwardOutputDim(conv_desc, input_desc, kernel_desc, &out_n, &out_c, &out_h, &out_w));

std::cout << "cuDNN 输出尺寸: " << out_h << "x" << out_w << "x" << out_c << std::endl;
const size_t output_data_size = (size_t)out_n * out_c * out_h * out_w;
CHECK_CUDA_ERR(cudaMalloc(&d_output, output_data_size * sizeof(float)));
CHECK_CUDNN_ERR(cudnnSetTensor4dDescriptor(output_desc, CUDNN_TENSOR_NCHW, CUDNN_DATA_FLOAT, out_n, out_c, out_h, out_w));

cudnnConvolutionFwdAlgoPerf_t perf_results[1];
int returned_algo_count;
CHECK_CUDNN_ERR(cudnnGetConvolutionForwardAlgorithm_v7(cudnn_handle, input_desc, kernel_desc, conv_desc, output_desc, perf_results, 1, CUDNN_CONVOLUTION_FWD_PREFER_FASTEST));
cudnnConvolutionFwdAlgo_t algo = perf_results[0].algo;
std::cout << "cuDNN 选择了算法: " << algo << std::endl;

size_t workspace_size = 0;
CHECK_CUDNN_ERR(cudnnGetConvolutionForwardWorkspaceSize(cudnn_handle, input_desc, kernel_desc, conv_desc, output_desc, &workspace_size));
void* d_workspace = nullptr;
if (workspace_size > 0) {
    CHECK_CUDA_ERR(cudaMalloc(&d_workspace, workspace_size));
}

std::cout << "cuDNN 工作空间大小: " << workspace_size / (1024.0 * 1024.0) << " MB" << std::endl;

CHECK_CUDA_ERR(cudaMemcpy(d_input, h_input, input_data_size * sizeof(float), cudaMemcpyHostToDevice));
CHECK_CUDA_ERR(cudaMemcpy(d_kernel, h_kernel, kernel_data_size * sizeof(float), cudaMemcpyHostToDevice));

cudaEvent_t start, stop;
CHECK_CUDA_ERR(cudaEventCreate(&start));
CHECK_CUDA_ERR(cudaEventCreate(&stop));

float alpha = 1.0f, beta = 0.0f;
CHECK_CUDA_ERR(cudaEventRecord(start));
CHECK_CUDNN_ERR(cudnnConvolutionForward(cudnn_handle, &alpha, input_desc, d_input, kernel_desc, d_kernel, conv_desc, output_desc, &beta, d_output));
CHECK_CUDA_ERR(cudaEventRecord(stop));
CHECK_CUDA_ERR(cudaDeviceSynchronize());

float milliseconds = 0;
CHECK_CUDA_ERR(cudaEventElapsedTime(&milliseconds, start, stop));
std::cout << "cuDNN 卷积计算时间: " << milliseconds << " ms" << std::endl;

```

3. 实验结果

任务 1:

```
ehpc@b1fbf95402d4: ~/Desktop$ ./mm 32 3
输入尺寸: 32x32x3
卷积核尺寸: 3x3x3
步幅: 1, 填充: 1
输出尺寸: 32x32x3
直接卷积计算时间: 0.18944 ms
ehpc@b1fbf95402d4: ~/Desktop$ ./mm 64 3
输入尺寸: 64x64x3
卷积核尺寸: 3x3x3
步幅: 1, 填充: 1
输出尺寸: 64x64x3
直接卷积计算时间: 0.205632 ms
ehpc@b1fbf95402d4: ~/Desktop$ ./mm 128 3
输入尺寸: 128x128x3
卷积核尺寸: 3x3x3
步幅: 1, 填充: 1
输出尺寸: 128x128x3
直接卷积计算时间: 0.219264 ms
ehpc@b1fbf95402d4: ~/Desktop$ ./mm 256 3
输入尺寸: 256x256x3
卷积核尺寸: 3x3x3
步幅: 1, 填充: 1
输出尺寸: 256x256x3
直接卷积计算时间: 0.464256 ms
ehpc@b1fbf95402d4: ~/Desktop$ ./mm 512 3
输入尺寸: 512x512x3
卷积核尺寸: 3x3x3
步幅: 1, 填充: 1
输出尺寸: 512x512x3
直接卷积计算时间: 1.15904 ms
```

```
ehpc@a12a68f8064a: ~/Desktop$ ./mm3 4096 3
输入尺寸: 4096x4096x3
卷积核尺寸: 3x3x3
步幅: 1, 填充: 1
输出尺寸: 4096x4096x3
直接卷积计算时间: 53.6147 ms
```

在优化方面，我尝试过使用共享内存进行优化，但优化效果反而更加不好，主要原因分析是其同步开销过大导致的。

任务 2:

```

ehpc@b1fbf95402d4: ~/Desktop$ ./mm2 32 3
输入尺寸： 32x32x3
卷积核尺寸： 3x3x3
步幅： 1, 填充： 1, 输出通道数： 3
输出尺寸： 32x32x3

GPU 端总计算时间 (im2col+GEMM): 0.163648 ms

ehpc@b1fbf95402d4: ~/Desktop$ ./mm2 64 3
输入尺寸： 64x64x3
卷积核尺寸： 3x3x3
步幅： 1, 填充： 1, 输出通道数： 3
输出尺寸： 64x64x3

GPU 端总计算时间 (im2col+GEMM): 0.204 ms

ehpc@b1fbf95402d4: ~/Desktop$ ./mm2 128 3
输入尺寸： 128x128x3
卷积核尺寸： 3x3x3
步幅： 1, 填充： 1, 输出通道数： 3
输出尺寸： 128x128x3

GPU 端总计算时间 (im2col+GEMM): 0.376352 ms

```

```

ehpc@b1fbf95402d4: ~/Desktop$ ./mm2 512 3
输入尺寸： 512x512x3
卷积核尺寸： 3x3x3
步幅： 1, 填充： 1, 输出通道数： 3
输出尺寸： 512x512x3

GPU 端总计算时间 (im2col+GEMM): 3.80758 ms

```

对比任务 1 的直接实现，似乎效果并不好。

在更大规模的问题上，实现效果也不如直接卷积：

```

ehpc@b1fbf95402d4: ~/Desktop$ nvcc mm2.cu -o mm2
ehpc@b1fbf95402d4: ~/Desktop$ ./mm2 4096 3
输入尺寸： 4096x4096x3
卷积核尺寸： 3x3x3
步幅： 1, 填充： 1, 输出通道数： 3
输出尺寸： 4096x4096x3

GPU 端总计算时间 (im2col+GEMM): 81.4756 ms

```

分析原因，问题应该是出在 im2col 身上，因为虽然我使用的 im2col 在 GPU 上进行优化，但是我的每次访问都是对全局内存进行的访问，并且由于 im2col 需要访问很多重复的数据，产生一个十分庞大的矩阵，这无疑会大大增加全局内存的访问次数，引入了巨大的中间数据开销，导致性能反而减低。

任务 3:

```
ehpc@a12a68f8064a: ~/Desktop$ ./mm 32 3
cuDNN 输出尺寸: 32x32x3
cuDNN 选择了算法: 0
cuDNN 工作空间大小: 0 MB
cuDNN 卷积计算时间: 0.135648 ms
ehpc@a12a68f8064a: ~/Desktop$ ./mm 64 3
cuDNN 输出尺寸: 64x64x3
cuDNN 选择了算法: 0
cuDNN 工作空间大小: 0 MB
cuDNN 卷积计算时间: 0.135936 ms
ehpc@a12a68f8064a: ~/Desktop$ ./mm 128 3
cuDNN 输出尺寸: 128x128x3
cuDNN 选择了算法: 0
cuDNN 工作空间大小: 0 MB
cuDNN 卷积计算时间: 0.194112 ms
ehpc@a12a68f8064a: ~/Desktop$ ./mm 256 3
cuDNN 输出尺寸: 256x256x3
cuDNN 选择了算法: 0
cuDNN 工作空间大小: 0 MB
cuDNN 卷积计算时间: 0.382432 ms
ehpc@a12a68f8064a: ~/Desktop$ ./mm 512 3
cuDNN 输出尺寸: 512x512x3
cuDNN 选择了算法: 0
cuDNN 工作空间大小: 0 MB
cuDNN 卷积计算时间: 0.990336 ms
ehpc@a12a68f8064a: ~/Desktop$ ./mm 4096 3
cuDNN 输出尺寸: 4096x4096x3
cuDNN 选择了算法: 0
cuDNN 工作空间大小: 0 MB
cuDNN 卷积计算时间: 56.0763 ms
```

神奇的是，在性能表现上，直接卷积和 cudnn 的表现差不多，cudnn 性能稍好于直接卷积。推测是因为通道数为 3 时，正好使得直接卷积高效利用了 L1 缓存，把通道数加大的结果如下，可以发现，此时 cudnn 展现出了绝对的优势，证明 cudnn 的优异表现。

```

ehpc@5b4424803076: ~/Desktop$ nvcc mm1.cu -o mm1 -lcudnn
ehpc@5b4424803076: ~/Desktop$ ./mm1 4096 3
cuDNN 输出尺寸: 4096x4096x
64
cuDNN 选择了算法: 0
cuDNN 工作空间大小: 0 MB
cuDNN 卷积计算时间: 118.431 ms
ehpc@5b4424803076: ~/Desktop$ nvcc mm2.cu -o mm2
ehpc@5b4424803076: ~/Desktop$ ./mm2 4096 3
输入尺寸: 4096x4096x3
卷积核尺寸: 3x3x3
步幅: 1, 填充: 1
输出尺寸: 4096x4096x64
直接卷积计算时间: 882.502 ms

```

把任务 2 的 im2col + GEMM 的通道数也加大，结果如下，看来之前 im2col + GEMM 在与直接卷积比较时落败的原因应该是因为直接卷积恰好高效利用了缓存的缘故：

```

ehpc@5b4424803076: ~/Desktop$ ./mm3 4096 3
输入尺寸: 4096x4096x3
卷积核尺寸: 3x3x3
步幅: 1, 填充: 1, 输出通道数: 64
输出尺寸: 4096x4096x64

GPU 端总计算时间 (im2col+GEMM): 420.929 ms

```

对比 im2col + GEMM 和 cudnn，会发现性能差距几乎是 4 倍，依旧是很大，

对于我实现的 im2col + GEMM 下一步的优化方向应该是吧 im2col 低效的数据访问进行优化，后续可以使用共享内存来进行进一步优化，感觉效果会好很多。

4. 实验感想

通过这次实验，我成功使用 cuda 编程对卷积操作进行优化，并且尝试了不同的方法，包括直接的卷积优化、im2col+GEMM 以及 cuDNN 的使用。并且对遇到的异常现象进行了分析，了解其背后的原因，通过这些实践，我对 cuda 编程更加熟悉，受益匪浅。